

Multiclass Network Intrusion Classification on the CIC-ToN-IoT Dataset Using Autoencoder and LightGBM

Final Project Proposal

(IF-47-INT)

103012340479

Adhyatma Chrysotomos Davu



Undergraduate Program Informatics

School of Computing

Telkom University

Bandung

2026

Approval Sheet

Multiclass Network Intrusion Classification on the CIC-ToN-IoT Dataset Using Autoencoder and LightGBM

Student ID: 103012340479
Adhyatma Chrysostomos Davu

This proposal is submitted as a proposed final project to
the Bachelor's Degree Program in Informatics
Faculty of Informatics, Telkom University

Bandung, 13 May 2026
Approved by

Prospective Supervisor 1

Prospective Supervisor 2

Agung Toto Wibowo
S.T., M.T., Ph.D.

Dr. I Wayan Palton Anuwiksa
S.Si., M.Si.

NIP: 06810035-1

NIP: 23950001

Abstract

The development of the *Internet of Things* (IoT) has made network traffic more diverse and has increased the potential for cyberattacks on interconnected devices. This condition encourages the use of a *Network-based Intrusion Detection System* (NIDS) that not only distinguishes normal and attack traffic, but also identifies attack types in more detail. This research aims to apply a combination of *Autoencoder* and *LightGBM* for multiclass network intrusion classification on the CIC-ToN-IoT dataset. The dataset is processed through *preprocessing* stages, including feature inspection, removal of unsuitable data or attributes, label encoding, stratified data splitting, and normalization using *MinMaxScaler*. The *Autoencoder* is used to reduce 67 network traffic features into a more compact latent representation, and the resulting latent features are then used as input for *LightGBM* to predict intrusion classes. Since the dataset has an imbalanced class distribution, this research also compares several *class imbalance* handling scenarios, namely no imbalance handling, *class weight*, *upsampling*, and *downsampling* on the training data. The CIC-ToN-IoT dataset is used with the *Attack* column as the class label, consisting of one normal class and nine attack classes. Model performance is evaluated using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*. The initial hypothesis of this research is that the latent representation produced by the *Autoencoder* can reduce feature dimensionality without removing the main patterns among classes, while *class imbalance* handling scenarios can affect the model's ability to recognize minority classes in multiclass classification on CIC-ToN-IoT.

Keywords: intrusion detection, CIC-ToN-IoT, autoencoder, LightGBM, multiclass classification, class imbalance.

Table of Contents

Abstract	i
Table of Contents	ii
List of Figures	iv
List of Tables	v
I Introduction	1
1.1 Background	1
1.2 Problem Formulation	2
1.3 Objectives	3
1.4 Hypotheses	3
1.5 Scope and Limitations	4
1.6 Research Plan	4
1.7 Research Schedule	6
II Literature Review	7
2.1 Intrusion Detection System	7
2.2 Feature Reduction and Dimensionality Reduction in Intrusion Detection System	11
2.3 Intrusion Detection System Dataset	18
2.4 Evaluation Metrics	19
III Methodology and System Design	24
3.1 System Flowchart	24
3.2 Research Dataset	27
3.3 Data Preprocessing	29
3.4 Dimensionality Reduction Using Autoencoder	30
3.5 Latent Feature Extraction	32
3.6 Experimental Scenarios	32
3.7 Classification Using LightGBM	33
3.8 Evaluation and Result Analysis	34
3.9 Algorithm	35

References	38
Appendix	41

List of Figures

2.1	Autoencoder architecture for dimensionality reduction	13
2.2	Illustration of multiclass confusion matrix	23
3.1	Proposed workflow for multiclass network intrusion classification using Autoencoder and LightGBM	25
3.2	Proposed Autoencoder and LightGBM architecture	31

List of Tables

1.1	Final project schedule	6
2.1	Comparison of IDS approaches	8
2.2	Comparison between HIDS and NIDS	9
2.3	Comparison between binary and multiclass classification in IDS	10
2.4	Comparison between feature selection and feature extraction . .	18
2.5	Class index mapping for multiclass evaluation on CIC-ToN-IoT	20
3.1	Dataset summary	28
3.2	Class distribution in CIC-ToN-IoT dataset	29
3.3	Preprocessing strategy	30
3.4	Proposed Autoencoder architecture	31
3.5	Feature transformation across modeling stages	32
3.6	Experimental scenarios for class imbalance handling	33
3.7	Main LightGBM configuration	34
3.8	Evaluation metrics	35

Chapter I

Introduction

Internet of Things (IoT) devices are increasingly used in household environments, industry, public services, and digital systems that rely on network connectivity. The growing number of connected devices makes network traffic more diverse in terms of volume, communication patterns, and types of activity. Under these conditions, an intrusion detection system should not only distinguish normal traffic from attack traffic in general, but also identify the type of attack in more detail. This research focuses on multiclass network intrusion classification using the CIC-ToN-IoT dataset by combining an *Autoencoder* as a latent feature extractor, *LightGBM* as the classification model, and *class imbalance* handling scenarios on the training data.

1.1 Background

The growth of IoT usage makes modern networks wider, more heterogeneous, and more difficult to monitor manually. Connected devices can generate traffic with different characteristics, while a security weakness in one device can be exploited to attack other services or devices. In this context, an *Intrusion Detection System* (IDS), particularly a *Network-based Intrusion Detection System* (NIDS), is used to recognize normal traffic patterns and traffic patterns that indicate attacks.

IDS research in IoT environments often uses network traffic data because such data can represent communication behavior among devices. The CIC-ToN-IoT dataset is one of the relevant datasets because it provides network traffic data with attack labels that can be used for multiclass classification. The dataset also presents several challenges commonly found in IDS research, such as a relatively large number of features, possible feature redundancy, and imbalanced class distribution [1, 2].

Multiclass classification on imbalanced data has its own challenges. A model may tend to learn the majority classes because they contain more samples, while minority classes become more difficult to recognize. In the context of intrusion detection, this condition can be problematic because several attack types may have only a small number of samples but still need to be detected by

the system. If the evaluation only relies on *accuracy*, the model performance may appear good even though the model is still weak in recognizing minority attack classes. Therefore, this research uses *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix* to assess performance at the class level.

Apart from class distribution, the data must also go through proper *pre-processing* so that the model does not learn misleading patterns. Identifier columns such as IP addresses, *flow ID*, or time can make the model depend on specific identities that appear only in the dataset, rather than on general network traffic patterns. Invalid values, features with very low variation, and differences in feature scales can also affect the training process. Incorrect *pre-processing* order, such as performing normalization before data splitting, may cause *data leakage* and make the evaluation results appear better than the actual condition [3, 4].

Network intrusion data can also have a relatively high feature dimension. Not all features contribute equally to the classification process. Some features may be redundant, less informative, or have non-linear relationships with other features. To reduce this complexity, *feature extraction* can be used to form a more compact new representation. In this research, an *Autoencoder* is used to reduce traffic features into a latent representation. This approach is selected because an *Autoencoder* can learn non-linear relationships among features through the data reconstruction process [5, 6, 7].

The latent representation produced by the *Autoencoder* is then used as input for *LightGBM*. The *LightGBM* model is selected because it is suitable for tabular data and is often used in NIDS research due to its efficient training and classification process [8, 9, 10]. With this design, *LightGBM* is not trained directly using the original features, but using the latent features produced through dimensionality reduction.

This research also compares several *class imbalance* handling scenarios, namely no imbalance handling, *class weight*, *upsampling*, and *downsampling*. These scenarios are applied to the training data to observe their effect on classification performance, especially on minority classes. The testing data is not modified so that the evaluation results still reflect the original distribution of the CIC-ToN-IoT dataset.

This research tests a multiclass network intrusion classification pipeline on CIC-ToN-IoT using *Autoencoder* and *LightGBM*. The dataset is prepared and normalized before latent features are extracted using the *Autoencoder*. The resulting latent features are then used to train *LightGBM* under several *class imbalance* handling scenarios. Model performance is analyzed using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*.

1.2 Problem Formulation

The problem formulations in this research are as follows:

1. How can the CIC-ToN-IoT dataset be prepared through *preprocessing* so that it can be used for multiclass network intrusion classification?
2. How can an *Autoencoder* be applied as a *feature extraction* method and *LightGBM* as a multiclass classification model based on the resulting latent features?
3. How does the model perform under several *class imbalance* handling scenarios based on *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*?

1.3 Objectives

The objectives of this research are as follows:

1. To prepare the CIC-ToN-IoT dataset through *preprocessing*, including feature selection, invalid data handling, label *encoding*, data splitting, and normalization.
2. To apply an *Autoencoder* to generate latent features and use those features as input for *LightGBM* in multiclass network intrusion classification.
3. To analyze model performance under several *class imbalance* handling scenarios using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*.

1.4 Hypotheses

The hypotheses in this research are formulated as follows:

1. A properly ordered *preprocessing* process, especially splitting the data before normalization, can reduce the risk of *data leakage* and produce data that is suitable for model training and testing.
2. The *Autoencoder* is expected to produce latent features that are more compact than the original CIC-ToN-IoT traffic features, while still allowing *LightGBM* to distinguish intrusion classes.
3. *Class imbalance* handling scenarios are expected to affect classification performance, especially on minority classes, making *macro* metrics and *confusion matrix* more informative than *accuracy* alone.

1.5 Scope and Limitations

The scope of this research is limited to the following:

1. The dataset used is CIC-ToN-IoT, with the *Attack* column used as the label for multiclass network intrusion classification.
2. This research focuses on the *Network-based Intrusion Detection System* (NIDS) approach, so the analyzed data consists of network traffic features, not host-based data or device system logs.
3. The *preprocessing* process includes removing unused columns, handling missing or invalid values, removing features with very low variation, label *encoding*, data splitting, and normalization using *MinMaxScaler*.
4. Dimensionality reduction is performed using an *Autoencoder*, and the classification model used is *LightGBM* with input in the form of latent features extracted by the *Autoencoder*.
5. *Class imbalance* handling is limited to four experimental scenarios, namely no imbalance handling, *class weight*, *upsampling*, and *downsampling*. All of these treatments are applied only to the training data.
6. Model performance evaluation is limited to *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*. This research does not discuss real-time IDS implementation on an actual network or IoT devices.

1.6 Research Plan

The research activities are arranged according to the final project workflow, starting from strengthening the theoretical foundation, preparing the dataset, designing the experimental pipeline, implementing the model, and evaluating the results.

1. Literature Study

The literature study is conducted to understand the concepts of IDS and NIDS, characteristics of IoT network data, the CIC-ToN-IoT dataset, feature reduction and dimensionality reduction, *Autoencoder*, *LightGBM*, *class imbalance*, and evaluation metrics for multiclass classification. The reviewed literature serves as the basis for determining the method and experimental design.

2. Literature Review

The literature review is compiled from previous studies related to IDS, *intrusion detection* datasets, *feature selection*, *feature extraction*, *Autoencoder*, *LightGBM*, *preprocessing*, *class imbalance* handling, and model evaluation. The discussion connects previous research with the experimental design proposed in this final project.

3. Data Collection and Inspection

Data collection is carried out by preparing the CIC-ToN-IoT dataset as the main data source for this research. The data is inspected in terms of the number of rows, number of columns, attribute types, class labels, usable features, and class distribution. This initial inspection becomes the basis for designing the *preprocessing* stage.

4. System Analysis and Design

System analysis and design include preparing the experimental workflow from dataset preparation to evaluation. The input features and target label are determined first, followed by the *preprocessing* strategy and data splitting design. After that, the normalization procedure, *Autoencoder* architecture, latent feature extraction mechanism, initial *LightGBM* configuration, *class imbalance* handling scenarios, and evaluation metrics are arranged as one experimental pipeline.

5. System Implementation

System implementation is carried out by building an experimental pipeline using the Python programming language. The data is processed according to the designed *preprocessing* steps, then the *Autoencoder* is trained to generate latent features. These latent features are used as input for *LightGBM* in each *class imbalance* handling scenario.

6. Implementation Result Analysis

Implementation result analysis is carried out by comparing predicted labels and actual labels on the testing data. Evaluation is performed using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*. The results from each scenario are compared to observe the effect of *class imbalance* handling on multiclass classification performance.

7. Report Writing

Report writing is carried out by documenting the background, literature review, methodology, experimental results, discussion, and conclusion. The report is prepared according to the applicable final project format and updated based on experimental results and supervisor feedback.

1.7 Research Schedule

The research schedule is arranged based on the planned sequence of activities, starting from the literature study to report writing. The schedule is presented in the form of a monthly *bar-chart* to show the duration and relationship among the work stages. The final project proposal schedule is shown in Table 1.1.

Table 1.1: Final project schedule

No	Activity	Month																							
		1				2				3				4				5				6			
1	Literature Study																								
2	Literature Re-view																								
3	Data Collection and Inspection																								
4	System Analysis and Design																								
5	System Implementation																								
6	Implementation Result Analysis																								
7	Report Writing																								

Chapter II

Literature Review

Research on network intrusion classification does not only depend on the model used, but also on how the data is understood, processed, reduced, and evaluated [1, 11]. In IoT network data, the number of features can be relatively large, the relationships among features are not always linear, and the class distribution is often imbalanced [2, 12]. These conditions make discussions on IDS, feature or dimensionality reduction, datasets, *preprocessing*, *class imbalance* handling, and evaluation metrics necessary as the foundation of this research [1, 6].

Several previous studies have discussed feature reduction, *feature extraction*, and the formation of new representations in *intrusion detection* data [13, 6, 5]. On the modeling side, *deep learning* approaches and variations of *Autoencoder* have been used to learn complex patterns in network data [14, 7]. Research on LightGBM, *ensemble learning*, and *feature selection* also provides a basis for selecting efficient models for tabular network data [10, 9, 15].

2.1 Intrusion Detection System

Intrusion Detection System (IDS) is used to recognize abnormal or malicious activities in networks or devices [11, 16]. In this research, IDS is discussed in the context of network traffic data because the developed model focuses on multiclass classification using network features from the CIC-ToN-IoT dataset [1, 2].

a. Definition of Intrusion Detection System

Intrusion Detection System (IDS) is a security system that monitors network or device activities to detect suspicious, abnormal, or malicious behavior [11, 16]. In modern networks, IDS helps distinguish *benign* traffic from *malicious* traffic so that attacks can be recognized more quickly [17, 18]. In IoT environments, IDS is especially relevant because connected devices are usually heterogeneous, numerous, and resource-constrained [18, 16].

Attack detection in IDS is commonly discussed through two approaches, namely *signature-based IDS* and *anomaly-based IDS* [11, 17]. *Signature-based IDS* matches traffic with known attack patterns, making this approach effec-

tive for documented attacks [11]. In contrast, *anomaly-based IDS* identifies deviations from normal patterns and is more suitable for recognizing *unknown attacks* or *zero-day attacks*, although the risk of *false positive* is usually higher [17, 12].

Table 2.1: Comparison of IDS approaches

IDS Approach	Characteristics	Limitations
<i>Signature-based IDS</i>	Detects attacks by matching network traffic with known attack patterns or signatures. This approach is effective for detecting attacks that have already been documented.	Less adaptive to new attacks, <i>unknown attacks</i> , or <i>zero-day attacks</i> because the attack patterns may not yet be available in the signature database.
<i>Anomaly-based IDS</i>	Detects attacks based on deviations from normal traffic behavior. This approach is more flexible for identifying previously unseen behavior.	May produce a higher <i>false positive rate</i> because not every deviation from normal behavior is an attack.

From the monitoring perspective, IDS can be categorized into *Host-based Intrusion Detection System* (HIDS) and *Network-based Intrusion Detection System* (NIDS) [14, 17]. HIDS monitors activities on a single host or device, while NIDS monitors traffic passing through a network [14]. Research based on network traffic datasets is closer to NIDS because the analyzed data consists of *flow*, packet, or statistical communication features [1, 11]. In IoT environments, the NIDS approach is relevant because one monitoring system can observe communication patterns across many devices without having to inspect logs from each device [11, 16].

Table 2.2: Comparison between HIDS and NIDS

IDS Type	Monitoring Focus	Relevance to This Research
<i>Host-based Intrusion Detection System</i> (HIDS)	Monitors activities on a specific host or device, such as system logs, running processes, or local device activities.	Less suitable for this research because the data used is not host log data, but network traffic features.
<i>Network-based Intrusion Detection System</i> (NIDS)	Monitors traffic passing through a network, such as <i>flow</i> , packet, or statistical communication features.	Suitable for this research because the classification process is based on network traffic features from the CIC-ToN-IoT dataset.

The development of NIDS no longer stops at binary detection between normal traffic and attack traffic, but also moves toward multiclass classification, improved *generalizability*, *class imbalance* handling, and model interpretability [1, 2]. In IoT-based IDS, models must also deal with complex attack patterns and imbalanced class distributions [12, 10]. Therefore, data-based IDS design needs to consider feature processing, model selection, and evaluation methods together [11, 6].

b. Classification in Intrusion Detection System

Classification in IDS determines the security label of network traffic [14, 17]. Input data can be mapped into a normal class, a general attack class, or more specific attack types [2, 10]. In IoT environments, this process becomes more challenging because communication patterns among devices can form more complex traffic variations [12]. In *intrusion detection* research, two commonly used scenarios are binary classification and multiclass classification [1, 14].

c. Classification Types

Binary classification divides data into two classes, namely normal traffic and attack traffic [1, 14]. This approach is often used when the research objective is only to determine whether traffic is safe or malicious [14]. Multiclass classification provides more detailed information because data is divided into several attack types [2, 10]. The challenge is greater because the model must distinguish patterns among attacks that may have similar characteristics [12, 10]. In this research, multiclass classification is selected because the research objective is not only to detect attacks, but also to identify attack types in CIC-ToN-IoT [1, 2].

Table 2.3: Comparison between binary and multiclass classification in IDS

Classification Type	Characteristics	Suitable Use
Binary classification	Divides data into two main classes, namely normal traffic and attack traffic. This approach is simpler because it only determines whether traffic is safe or malicious.	Suitable for initial detection when the main goal is to distinguish normal traffic from attack traffic in general.
Multiclass classification	Divides data into more specific classes, such as normal, DDoS, DoS, Backdoor, Injection, and other attack types. This approach is more complex because the model must distinguish patterns among different attack types.	Suitable for this research because the classification goal is not only to detect attacks, but also to identify attack types in the CIC-ToN-IoT dataset.

d. Machine Learning Models

Machine learning models are widely used in IDS because network traffic data is generally tabular [1, 6]. Tree-based models such as *Decision Tree*, *Random Forest*, *Balanced Random Forest*, and *Extra Trees* are often used because they can handle numerical and categorical features after *preprocessing* [6, 10]. Other models such as *Naive Bayes*, *K-Nearest Neighbors*, *Support Vector Machine*, *Logistic Regression*, and *Multi-Layer Perceptron* also appear in NIDS research as performance comparisons [3, 17].

The *boosting* group, such as LightGBM, XGBoost, and CatBoost, is widely used for tabular data because it can build predictive models from a combination of many decision trees [2, 10]. LightGBM is also often combined with optimization or *feature selection* techniques to improve detection efficiency and performance [9, 15]. In this research, LightGBM is selected because it is suitable for the tabular characteristics of features extracted by the *Autoencoder* [8, 9].

A *gradient boosting* model forms predictions by summing the outputs of several decision trees [8, 9]. Its general form can be written as follows.

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad (2.1)$$

In Equation (2.1), $\hat{y}_i$ is the prediction result for the i -th sample, while x_i represents the input data for the same sample [8]. The model consists of K

decision trees, and $f_k(x_i)$ represents the output of the k -th tree for data x_i [9]. The index i refers to the order of data samples, not attack classes. Meanwhile, the index k indicates the order of trees in the *gradient boosting* model [8].

e. Deep Learning Models

Deep learning models are used in IDS when the data has complex patterns or relationships among features that are difficult to capture using simpler models [14, 19]. *Deep Neural Network* (DNN) is often used for tabular data, while *Convolutional Neural Network* (CNN) is used to capture local patterns in certain data representations [14, 20]. Sequential models such as *Long Short-Term Memory* (LSTM), *Gated Recurrent Unit* (GRU), and *attention-based Bi-LSTM* are selected when traffic sequences or dynamics are part of the analysis [19, 21].

In IDS research, *Autoencoder* is often placed before the classifier to strengthen feature representation [7, 22]. This model is not always used as the main classifier, but as a *feature extraction* stage that generates latent representations from original features [6, 7]. This position is consistent with the design of this research, because the *Autoencoder* is used to generate latent features before classification using *LightGBM* [7, 9].

2.2 Feature Reduction and Dimensionality Reduction in Intrusion Detection System

Network traffic data may contain many features, but not all features provide the same information for the classification process [13, 6]. Some features may be redundant, have low variation, or contribute little to distinguishing attack classes [2, 10]. Feature reduction and dimensionality reduction are used to reduce data complexity, lower computational cost, and help the model focus on more relevant patterns [13, 6].

a. Objectives of Reduction

Reduction in IDS aims to reduce data complexity, accelerate training and testing, suppress feature redundancy, and help models learn more relevant attack patterns [13, 6]. These goals are closely tied to network intrusion datasets, which may contain many traffic attributes and large numbers of records [1, 2]. With more compact features, the modeling process can run more efficiently without always using all original attributes [6, 10].

b. Feature Selection

Feature selection selects a subset of original features that are considered most relevant for classification [13, 10]. Statistical methods such as *minimum Redundancy Maximum Relevance* (mRMR), Pearson Correlation, and Chi-square are used to assess the relationship between features and the target or relationships among features [2, 6]. *Model importance*-based approaches use models such as *Extra Tree Classifier*, *Random Forest*, XGBoost, LightGBM, or *Recursive Feature Elimination* (RFE) to determine features that influence prediction [14, 10].

In addition to those two approaches, several studies use hybrid or optimization methods [9, 16]. Examples include *Grey Wolf Optimization* (GWO), *Particle Swarm Optimization* (PSO), and *Harris Hawk Optimization* (HHO) [9, 16]. The advantage of *feature selection* lies in its interpretability because the retained features are still original features [13, 10]. However, this method does not always capture non-linear relationships among features, especially when attack patterns are distributed across combinations of several attributes [13, 6].

c. Feature Extraction

Feature extraction does not directly select original features, but forms a new representation from the initial features [13, 6]. In IDS, this approach includes linear methods such as *Principal Component Analysis* (PCA) and neural network-based methods such as *Autoencoder* (AE), *Sparse Autoencoder* (SAE), and *Stacked Sparse Autoencoder* (SSAE) [6, 23, 24]. Other variations such as *One-Class Autoencoder* are used for anomaly detection in IoT networks [22]. *Autoencoder* representations are also often used before the classification stage in network intrusion data [7, 20].

PCA projects data into a new space while preserving the largest variance in the data [6]. Its projection form can be written as follows.

$$\mathbf{z} = W^T(\mathbf{x} - \boldsymbol{\mu}) \quad (2.2)$$

In Equation (2.2), the original feature vector is denoted by $\mathbf{x}$, while $\boldsymbol{\mu}$ is the feature mean used for data centering [6]. The matrix W^T contains the selected principal components, and the projection result is denoted by $\mathbf{z}$. Through this projection, the original data can be represented in a lower dimension [6].

Autoencoder works by learning a latent representation of data and then attempting to reconstruct the data [5, 6]. In dimensionality reduction, the most relevant part is the latent representation located in the middle of the architecture [7, 22]. This representation is expected to store the main information from the original features in a more compact form [6, 23]. A simple form of the *encoder* process can be written as follows.

$$\mathbf{h} = f(W_e \mathbf{x} + b_e) \quad (2.3)$$

Equation (2.3) shows the process of forming a latent representation. The input data $\mathbf{x}$ is multiplied by the *encoder* weight W_e , then added with the bias b_e . The activation function $f(\cdot)$ produces the representation $\mathbf{h}$ as the output of the *encoding* process. This representation is used as a compact form of the input data [5, 6].

After the latent representation is formed, the *decoder* attempts to reconstruct the data from that representation [5, 7]. This reconstruction process helps the model learn whether the information stored in the latent space is

sufficient to represent the original data [6, 22]. A simple form of the *decoder* process is written as follows.

$$\hat{\mathbf{x}} = g(W_d \mathbf{h} + b_d) \quad (2.4)$$

In Equation (2.4), the latent representation $\mathbf{h}$ is mapped back using the *decoder* weight W_d and bias b_d . The activation function $g(\cdot)$ produces $\hat{\mathbf{x}}$, which is the reconstructed version of the input data. The closer $\hat{\mathbf{x}}$ is to $\mathbf{x}$, the better the latent representation is at preserving patterns from the original data [5, 6].

The training objective of an *Autoencoder* is to reduce the difference between the original data and the reconstructed data [5, 6]. One form of the reconstruction error function can be written as follows.

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 \quad (2.5)$$

The value $\mathcal{L}$ in Equation (2.5) represents the reconstruction error. The difference between $\mathbf{x}$ and $\hat{\mathbf{x}}$ is calculated using the squared L2 norm, so larger differences give a greater penalty during training [5, 6]. A similar approach is also used in hybrid models that utilize *Autoencoder* representations before classification or anomaly detection [7, 22].

In an *Autoencoder*, model parameters can be denoted by the symbol θ [5, 6]. These parameters include weights and biases in the *encoder* and *decoder*. In a simple form, the parameters can be written as follows.

$$\theta = \{W_e, b_e, W_d, b_d\} \quad (2.6)$$

In Equation (2.6), W_e and b_e represent the weight and bias in the *encoder*, while W_d and b_d represent the weight and bias in the *decoder*. During training, the parameter θ is updated so that the reconstructed data becomes closer to the input data. The value θ^* represents the best parameter obtained after optimizing the reconstruction error function [5, 6].

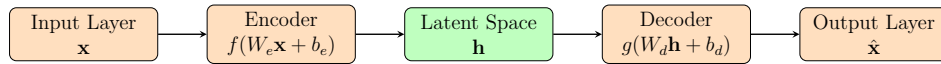


Figure 2.1: Autoencoder architecture for dimensionality reduction

Figure 2.1 shows the general architecture of an *Autoencoder*, where the input data $\mathbf{x}$ is processed by the *encoder* to form the latent representation $\mathbf{h}$ [5, 6]. This representation is then used by the *decoder* to produce the reconstructed data $\hat{\mathbf{x}}$ [7, 22]. In the context of dimensionality reduction, the latent representation is the most relevant part because it stores a compact form of the input data [6, 23].

d. Data Normalization and MinMaxScaler

Data normalization is performed so that features with different value ranges can be processed on a more uniform scale [4, 6]. In network traffic data, some features can have large values, such as flow duration or byte count, while other features can have small values, such as flags or protocols [1, 2]. These scale differences can affect the learning process, especially in neural network-based models such as *Autoencoder* [6, 7]. Normalization helps the training process become more stable because each feature is placed within a comparable range [4, 6].

One commonly used normalization method is *MinMaxScaler* [4, 6]. This method transforms feature values into a certain range, usually between 0 and 1. The transformation can be written as follows.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (2.7)$$

In Equation (2.7), the original feature value is denoted by x . The minimum and maximum values of the feature are denoted by $x_{\min}$ and $x_{\max}$, while the normalized result is denoted by x' [4]. In the experimental design, the minimum and maximum values should be calculated only from the training data, then used to transform both the training and testing data [3, 4]. This prevents the testing data from influencing the *preprocessing* parameters during model training [3, 4].

e. Data Preprocessing in Intrusion Detection System

Preprocessing in IDS research is carried out so that network traffic data is in a format that can be processed by the model [3, 4]. In network data, common issues include missing values, infinite values, inappropriate data types, features with very low variation, and metadata columns that do not represent traffic patterns [1, 4]. This stage must be performed carefully because incorrect ordering, such as normalizing data before splitting, can cause *data leakage* and make the evaluation results appear better than the actual condition [3, 4].

Data can be considered invalid when its values cannot be processed numerically or no longer match the meaning of the attribute [4]. Examples include *NaN* values, infinite values such as *inf* and *-inf*, strings appearing in numerical columns, or conversion results that cannot be interpreted as network traffic features [3, 4]. If invalid values appear only in a small number of rows and cannot be logically corrected, those rows can be removed [4]. However, if a feature contains too many invalid values or has almost no variation, the feature is more appropriate to remove from the input data because its contribution to classification is very small [13, 6].

Identifier columns also need to be examined before being used as features [1, 3]. Columns such as *Flow ID*, *Src IP*, *Dst IP*, and *Timestamp* can make the model depend on specific identities that appear in the dataset, rather than general network traffic patterns [3, 4]. In the NIDS context, more relevant

features are traffic statistics, such as packet length, flow duration, packet count, flags, protocol, port, and other communication characteristics that describe traffic behavior [1, 11].

f. Class Imbalance Handling

Class imbalance occurs when the number of samples in each class is unequal [2, 10]. In *intrusion detection* datasets, this condition often appears because some attack types have far fewer samples than normal traffic or certain attack types [1, 2]. If this condition is not handled, the model may more easily learn the majority classes and fail to recognize minority classes [10, 9]. As a result, the *accuracy* value may appear high, while *macro recall* or *macro F1-score* for minority classes remains low [2, 10].

One approach to handling *class imbalance* is *class weight* [2, 10]. In this approach, classes with fewer samples are assigned larger weights during training [9, 15]. Class weights can be calculated using the following formula.

$$w_c = \frac{n}{C \cdot n_c}, \quad \forall c \in \{1, 2, \dots, C\} \quad (2.8)$$

In Equation (2.8), w_c is the weight for class c , n is the total number of training samples, C is the number of classes, and n_c is the number of training samples in class c [2, 10]. The fewer the samples in a class, the larger the weight assigned to that class [9, 15].

Other approaches include *upsampling* and *downsampling* [2, 10]. *Upsampling* is performed by increasing the number of samples in minority classes, for example through sample duplication or synthetic sample generation [10, 9]. In contrast, *downsampling* is performed by reducing the number of samples in majority classes so that the training data distribution becomes more balanced [2, 10]. In a valid experiment, *upsampling* and *downsampling* are applied only to the training data, while the testing data is not modified so that the evaluation still represents the original dataset distribution [3, 4].

g. Notation and Components in the Autoencoder-LightGBM Pipeline

The notation used in the proposed pipeline follows the main stages of the experiment: *preprocessing*, *Autoencoder* training, latent feature extraction, *class imbalance* handling, *LightGBM* training, and model evaluation [6, 8]. These notations allow the algorithm in Chapter III to focus on the workflow without repeating every technical explanation [5, 9].

Input data that has gone through *preprocessing* and normalization is denoted by X_{train}^{norm} and X_{test}^{norm} [3, 4]. The notation X_{train}^{norm} represents normalized training features using parameters from the training data, while X_{test}^{norm} represents testing features normalized using the same parameters [4]. Labels for the training and testing data are denoted by y_{train} and y_{test} . This notation emphasizes that normalization is performed after data splitting, so the testing data does not influence the *preprocessing* parameters [3, 4].

At the *Autoencoder* stage, the model parameters are denoted by θ [5, 6]. These parameters include weights and biases in the *encoder* and *decoder*, namely $\theta = \{W_e, b_e, W_d, b_d\}$. During training, the parameter θ is updated to minimize the reconstruction error between the input data and reconstructed data [5, 7]. The best trained parameter is denoted by θ^* . This optimization process can be written as follows.

$$\theta^* = \arg \min_{\theta} \|X_{train}^{norm} - \text{Decoder}_{\theta}(\text{Encoder}_{\theta}(X_{train}^{norm}))\|_2^2 \quad (2.9)$$

In Equation (2.9), the *encoder* transforms input data into a latent representation, while the *decoder* attempts to reconstruct the data from that representation [5, 6]. The reconstruction error is calculated from the difference between X_{train}^{norm} and the reconstruction result produced by the *decoder* [7, 22]. The smaller the reconstruction error, the better the *Autoencoder* is at forming latent representations that retain patterns from the input data [6, 23].

After the *Autoencoder* is trained, the *encoder* is used to generate latent features [7, 6]. The latent features from the training and testing data are denoted by Z_{train} and Z_{test} . The latent feature extraction process can be written as follows.

$$Z_{train} = \text{Encoder}_{\theta^*}(X_{train}^{norm}), \quad Z_{test} = \text{Encoder}_{\theta^*}(X_{test}^{norm}) \quad (2.10)$$

In Equation (2.10), Z_{train} is used as input to train *LightGBM*, while Z_{test} is used as input in the prediction stage [8, 9]. Thus, the classification model does not work directly on the original features, but on the latent representation produced through dimensionality reduction [6, 7].

Class imbalance handling in this research is represented by scenario S [2, 10]. The scenarios include no imbalance handling, *class weight*, *upsampling*, and *downsampling* [10, 9]. In the *class weight* scenario, the weight for each class follows Equation (2.8), where classes with fewer samples receive larger weights during training [2, 15]. In the *upsampling* scenario, the number of minority class samples in the training data is increased, while in the *downsampling* scenario, the number of majority class samples in the training data is reduced [2, 10]. These treatments are applied only to the training data, while the testing data remains in its original distribution [3, 4].

The trained *LightGBM* model is denoted by M_{LGBM} [8, 9]. This model receives the latent features Z_{train} and labels y_{train} as training inputs [15, 10]. After training is completed, the model is used to generate predictions on the latent features of the testing data [8]. The predicted class is denoted by $\hat{y}_{test}$ and can be written as follows.

$$\hat{y}_{test} = M_{LGBM}(Z_{test}) \quad (2.11)$$

The prediction result $\hat{y}_{test}$ is then compared with the actual label y_{test} during evaluation [14, 10]. The evaluation produces *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix* values [2, 14]. With this notation, the algorithm in Chapter III can focus on the main workflow without repeatedly explaining the concepts of *preprocessing*, *Autoencoder*, *class imbalance*, *LightGBM*, and evaluation metrics in detail [6, 8].

h. Comparative Analysis of Feature Selection and Feature Extraction

Feature selection and *feature extraction* offer different ways of performing reduction [13, 6]. *Feature selection* is easier to trace because the features used remain original attributes [13, 10]. This approach is suitable when interpretability and efficiency are the main concerns [13]. *Feature extraction* produces new features through a transformation process, so its interpretation is not always direct [6, 23]. However, this approach is more flexible for high-dimensional data, non-linear patterns, and multiclass classification because the new representation can capture relationships among features that may not be visible in the original features [6, 22]. In this research, *Autoencoder* is selected because the main objective is to generate latent features before classification using *LightGBM* [7, 9].

Table 2.4: Comparison between feature selection and feature extraction

Aspect	<i>Feature Selection</i>	<i>Feature Extraction</i>
Working principle	Selects a subset of original features that are considered most relevant to the classification process.	Forms a new lower-dimensional representation from the original features.
Output form	A subset of original features that still preserves the meaning of the initial attributes.	New features or latent features produced through transformation of the original features.
Interpretability	Easier to interpret because the selected features are still original features.	More difficult to interpret because the resulting features are transformed representations.
Advantages	Computationally lighter, easier to understand, and useful for reducing irrelevant features.	More capable of capturing complex patterns and non-linear relationships in high-dimensional data.
Limitations	Depends on the quality of the feature selection method and may miss non-linear relationships between features.	Requires an additional training or transformation process, and the resulting representation is not always easy to explain.
Research relevance	Used as a conceptual comparison for feature reduction in IDS.	More relevant to this research because <i>Autoencoder</i> is used to generate latent features before classification using LightGBM.

2.3 Intrusion Detection System Dataset

A dataset determines what patterns can be learned by an IDS model [1, 11]. If the data does not represent network traffic well, the model evaluation results will also be difficult to use as a basis for conclusions [1]. A dataset for *intrusion detection* needs to contain normal and attack traffic, have relevant features, and support fair evaluation across models [1, 11]. In the context of IoT and IoMT, variations in devices, traffic types, and attack forms also need to be considered because connected device environments have different characteristics from general networks [16, 18].

Various datasets are used in *intrusion detection* research, such as UNSW-NB15, NSL-KDD, CICIDS2017, BoT-IoT, TON-IoT, CICIoT2023, IoTID20, and CICIoMT2024 [14, 22, 20]. These datasets are used to test *machine learn-*

ing and *deep learning* models under various network scenarios [14, 21]. In other studies, IDS datasets are used to test hybrid approaches, *feature selection*, and classification models in IoT environments [17, 21, 15]. One challenge is that not all datasets provide a uniform *feature set*, making results across studies not always easy to compare [1].

This research uses CIC-ToN-IoT because the dataset provides a *common feature set* based on CICFlowMeter, allowing experiments to be conducted with more consistent features [1]. Nevertheless, CIC-ToN-IoT still has challenges, especially feature redundancy, *class imbalance*, and uneven performance across attack types [2]. The complex characteristics of IoT traffic can also make intrusion classification more difficult than classification on general network data [12]. For these reasons, CIC-ToN-IoT is suitable for testing the dimensionality reduction and multiclass classification pipeline in this research [1, 2].

2.4 Evaluation Metrics

IDS model evaluation cannot rely only on the overall number of correct predictions [1, 2]. In imbalanced data, a model can obtain high *accuracy* because it successfully recognizes majority classes, while still failing to recognize minority classes [2, 10]. This issue often appears in *intrusion detection* because the number of samples across attack types can differ greatly [1, 10]. Therefore, this research uses *accuracy*, *macro-based metrics*, and *confusion matrix* so that model performance can be observed from both overall and per-class perspectives [10, 14].

In the formulas for multiclass evaluation metrics, the index i is used to represent the i -th class, while C represents the total number of classes used in the classification process [14, 2]. Since this research uses one normal class and nine attack classes, $C = 10$. The class index mapping used in the evaluation is shown in Table 2.5.

Table 2.5: Class index mapping for multiclass evaluation on CIC-ToN-IoT

Index	Class	Description
$i = 1$	<i>Benign</i>	Normal traffic that does not contain attack activity.
$i = 2$	<i>Backdoor</i>	An attack that aims to create hidden access to a system.
$i = 3$	<i>DDoS</i>	A distributed attack that aims to overwhelm a service or network with excessive traffic.
$i = 4$	<i>DoS</i>	An attack that aims to disrupt service availability from a single source or a specific attack pattern.
$i = 5$	<i>Injection</i>	An attack that inserts malicious commands or inputs into a system.
$i = 6$	<i>MITM</i>	A <i>Man-in-the-Middle</i> attack that aims to intercept or manipulate communication between parties.
$i = 7$	<i>Password</i>	An attack related to attempts to obtain or guess user credentials.
$i = 8$	<i>Ransomware</i>	An attack that aims to encrypt or lock victim data for ransom.
$i = 9$	<i>Scanning</i>	Reconnaissance activity used to identify services, ports, or vulnerabilities in a network.
$i = 10$	<i>XSS</i>	A <i>Cross-Site Scripting</i> attack that inserts malicious scripts into web applications or web communication.

a. Accuracy

Accuracy measures the proportion of correct predictions over all data [14, 2]. This metric is useful for observing general performance, but it is not sufficient when used alone on imbalanced data [1, 10].

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.12)$$

In Equation (2.12), TP and TN indicate correct predictions for positive and negative classes, while FP and FN indicate incorrect predictions [14]. In multiclass classification, these values can be calculated using a *one-vs-rest* approach, where one class is treated as the positive class and the other classes are treated as negative classes [2, 10].

b. Macro Precision

Precision measures how many positive predictions are correct [14, 2]. In multiclass classification, *macro precision* is calculated by averaging the precision values of all classes [10, 14]. Each class is given the same weight, so minority classes still influence the final value [2, 10].

$$\text{Macro Precision} = \frac{1}{C} \sum_{i=1}^C \text{Precision}_i \quad (2.13)$$

In Equation (2.13), C represents the number of classes, while Precision_i is the precision value for the i -th class [14]. The precision value for each class is calculated using the following formula.

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i} \quad (2.14)$$

In Equation (2.14), TP_i is the number of data from the i -th class correctly predicted as that class [14]. Meanwhile, FP_i is the number of data from other classes incorrectly predicted as the i -th class. This value indicates how precise the model is when assigning predictions to a certain class [2, 10].

c. Macro Recall

Recall measures the model's ability to find data that truly belongs to a certain class [14, 10]. In this research, *macro recall* is used so that detection ability for each class can be observed without being dominated by the number of majority class samples [2, 10].

$$\text{Macro Recall} = \frac{1}{C} \sum_{i=1}^C \text{Recall}_i \quad (2.15)$$

Equation (2.15) calculates the average recall value of all classes [14, 2]. The recall value for the i -th class is calculated as follows.

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i} \quad (2.16)$$

In Equation (2.16), TP_i is the number of data from the i -th class that are correctly recognized, while FN_i is the number of data from the i -th class incorrectly predicted as another class [2, 10]. A low recall value for a certain class indicates that the model still has difficulty recognizing that class [10, 14].

d. Macro F1-score

F1-score combines *precision* and *recall* into a single value [14, 2]. In multiclass classification, *macro F1-score* is used to observe the balance of model performance across all classes [2, 10].

$$F1_i = \frac{2 \times \text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \quad (2.17)$$

Equation (2.17) calculates the F1-score for the i -th class by combining Precision_i and Recall_i [14]. If either value is low, then $F1_i$ will also decrease [2, 10]. After the F1-score for each class is obtained, the *macro F1-score* is calculated as follows.

$$\text{Macro F1-score} = \frac{1}{C} \sum_{i=1}^C F1_i \quad (2.18)$$

In Equation (2.18), $F1_i$ represents the F1-score value for the i -th class, while C is the number of classes [2, 14]. Since it uses a simple average, each class contributes equally to the final value [10, 2].

e. Confusion Matrix

In addition to single-value metrics, this research also uses a *confusion matrix* [14, 17]. This matrix shows the number of correct and incorrect predictions for each class [14]. In binary classification, a *confusion matrix* consists of TP , TN , FP , and FN [17]. In multiclass classification, rows represent actual classes and columns represent predicted classes [14, 2].

As shown in Figure 2.2, the multiclass *confusion matrix* is read by comparing the actual class on the vertical axis with the predicted class on the horizontal axis [14, 17]. Each row represents the actual class of the data, while each column represents the class predicted by the model [14]. The diagonal cells represent correct predictions because the actual class and the predicted class are the same. In contrast, the cells outside the diagonal represent misclassifications [17].

Good classification results usually appear as high values along the diagonal and low values outside the diagonal [14, 2]. In intrusion detection, this analysis is useful because a model may have high overall accuracy but still misclassify certain attack types [1, 10]. By observing the *confusion matrix*, the evaluation can show not only the number of correct predictions, but also the distribution of errors across different attack classes [14, 17].

Actual Class	Benign	95	0	1	1	0	0	1	0	2	0
	Backdoor	1	88	0	0	2	1	4	1	2	1
	DDoS	0	0	92	5	0	0	0	0	3	0
	DoS	0	0	6	89	1	0	0	0	4	0
	Injection	0	2	0	1	86	0	3	0	1	7
	MITM	0	1	0	0	0	91	1	0	6	1
	Password	1	4	0	0	3	0	87	1	2	2
	Ransomware	0	1	0	0	0	0	2	93	3	1
	Scanning	2	1	4	3	0	2	1	0	87	0
	XSS	0	1	0	0	6	0	2	0	1	90
		Predicted Class									
		Benign	Backdoor	DDoS	DoS	Injection	MITM	Password	Ransomware	Scanning	XSS

Figure 2.2: Illustration of multiclass confusion matrix

The combination of *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix* is used so that evaluation does not depend on a single number only [1, 2]. *Accuracy* shows overall performance, *macro* metrics give equal attention to each class, and *confusion matrix* helps identify error patterns among classes [10, 14].

Chapter III

Methodology and System Design

This research is designed as a multiclass network intrusion classification experiment using the CIC-ToN-IoT dataset. The research workflow includes dataset preparation, *preprocessing*, dimensionality reduction using an *Autoencoder*, classification using *LightGBM*, *class imbalance* handling scenarios, and model performance evaluation. The workflow is arranged so that the testing data does not influence the training process, especially during normalization, *Autoencoder* training, and classifier training. This methodological design refers to the need to maintain dataset quality, *preprocessing* consistency, feature or dimensionality reduction, and appropriate evaluation for network intrusion data [1, 3]. Several studies also emphasize that NIDS experiments need to consider the order of *preprocessing* and the selection of feature reduction methods so that model performance does not appear overly optimistic due to *data leakage* [4, 6].

The CIC-ToN-IoT dataset is used with the *Attack* column as the class label. Based on the initial inspection, the dataset contains 5,351,760 records, 85 columns, and 10 classes. After the feature inspection process, 67 numerical features are used as model input, while the complete list of features is placed in the Appendix. These features are processed using an *Autoencoder* to generate a lower-dimensional latent representation. The latent representation is then used as input for *LightGBM* to perform multiclass classification. This combination is used to examine whether the latent feature representation produced by the *Autoencoder* can help *LightGBM* distinguish normal classes and various attack types in CIC-ToN-IoT.

3.1 System Flowchart

The system flowchart illustrates the workflow of the proposed model in this research. The dataset loading stage, feature and label identification, and feature screening are combined into the *Load and Prepare CIC-ToN-IoT Dataset* stage. The *preprocessing* stage includes invalid data inspection, removal of unused features, label *encoding*, data splitting, and normalization. With this arrangement, the flowchart becomes more concise without removing the tech-

nical processes required before model training.

After the data is processed, the *Autoencoder* is trained to generate latent features. The latent features from the training and testing data are then used in the classification stage. Before *LightGBM* is trained, this research adds a *class imbalance* handling scenario to the training data. The final stage consists of intrusion class prediction and evaluation using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*. The system flowchart is shown in Figure 3.1.

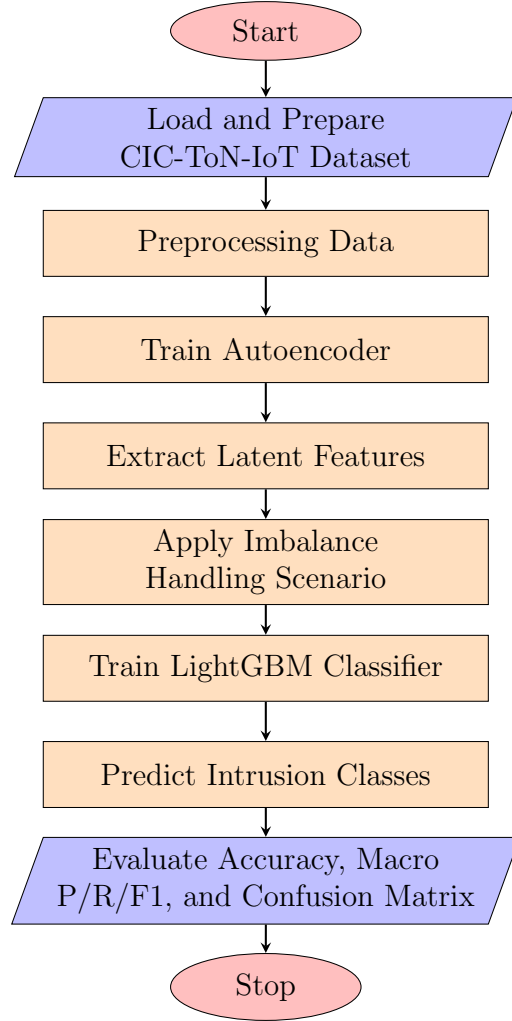


Figure 3.1: Proposed workflow for multiclass network intrusion classification using Autoencoder and LightGBM

Explanation of the System Flowchart Stages

The workflow in Figure 3.1 starts from dataset preparation and ends with prediction result evaluation. Each stage produces an output that is used by

the next stage, so the experimental process can be traced from the initial data to the final evaluation results.

a. Load and Prepare CIC-ToN-IoT Dataset

This stage includes dataset loading, feature and label identification, and initial inspection of the data structure. The CIC-ToN-IoT dataset is loaded into the data processing environment, then inspected based on the number of rows, number of columns, attribute names, data types, and available class labels. The *Attack* column is used as the main label because it contains the normal class and attack types. Other columns are inspected as feature candidates based on their data type, attribute meaning, and value variation.

Columns such as *Flow ID*, *Src IP*, *Dst IP*, and *Timestamp* are not used as model features because they act as identifiers or metadata. These columns may cause the model to learn specific traffic identities rather than general network traffic patterns. *Quasi-constant* features are also not used because their variation is very low and they provide little assistance in separating classes. The output of this stage is a set of numerical features suitable for further processing and a determined class label.

b. Preprocessing Data

The *preprocessing* stage prepares the data so that it can be used by the *Autoencoder* and *LightGBM*. This process includes removing unused columns, checking missing values, checking invalid values, adjusting data types, encoding labels, splitting data, and normalizing features. Data is considered invalid when the values in numerical features cannot be processed by the model, such as *NaN*, infinite values, strings in numerical columns, or converted values that no longer have meaning with respect to the original attribute.

If invalid values appear only in a small number of rows and cannot be logically corrected, those rows can be removed. If a feature contains too many missing values, infinite values, or very low variation, the feature can be removed from the input data. If the class label is missing or does not match the classes used in this research, the corresponding data is not used because the classification target is invalid. After the data is cleaned, the labels in the *Attack* column are encoded into numerical form.

Data splitting is performed using *stratified split* with an 80:20 ratio. Stratification is used so that the class proportion in the training and testing data remains close to the original class distribution in the dataset. After data splitting, normalization is performed using *MinMaxScaler*. The normalization parameters are calculated only from the training data, then the same parameters are used to transform both the training and testing data. This order is used to prevent information from the testing data from influencing the training process.

c. Train Autoencoder

The *Autoencoder* is trained using the normalized training data. This model

learns a compact representation of the original features through the *encoder* part, while the *decoder* part attempts to reconstruct the data from that representation. Training is performed by minimizing the reconstruction error between the input data and the reconstructed data. The *Autoencoder* parameters are denoted by θ , which includes the weights and biases in the *encoder* and *decoder*.

d. Extract Latent Features

After the *Autoencoder* has been trained, the *encoder* part is used to extract latent features from the training and testing data. In this research, the input features are reduced into 16 latent features. The latent features from the training data are used to train *LightGBM*, while the latent features from the testing data are used for prediction. This stage connects dimensionality reduction and classification because the classifier does not use the original features directly.

e. Apply Imbalance Handling Scenario

This stage is applied to the latent features of the training data and the training labels. The testing data is not modified so that the evaluation still reflects the original dataset distribution. This research uses several *class imbalance* handling scenarios, namely no imbalance handling, *class weight*, *upsampling*, and *downsampling*. These scenarios are used to observe the effect of imbalance handling on multiclass classification performance.

f. Train LightGBM Classifier

LightGBM is trained using the latent features of the training data as input and the class labels as the target output. This model learns the relationship between the latent features produced by the *Autoencoder* and the network intrusion classes. Since the input to *LightGBM* consists of latent features, the classification stage is performed on the reduced-dimensional representation rather than directly on the original features.

g. Predict Intrusion Classes

After *LightGBM* has been trained, the model is used to predict intrusion classes on the latent features of the testing data. The prediction results are class labels, either normal traffic or specific attack types. The predicted labels are then compared with the actual labels to determine model performance.

h. Evaluate Accuracy, Macro P/R/F1, and Confusion Matrix

Evaluation is performed using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*. *Accuracy* is used to observe overall performance, while *macro* metrics are used to examine the performance of each class with equal weighting. The *confusion matrix* is used to observe the distribution of correct and incorrect predictions across classes.

3.2 Research Dataset

This research uses CIC-ToN-IoT as the main dataset for IoT traffic-based multiclass network intrusion classification. In NIDS research, network traf-

fic features and the consistency of the *feature set* influence the reliability of model evaluation [1, 11]. Based on the initial inspection, the dataset contains 5,351,760 rows and 85 columns. The *Attack* column is used as the class label because it contains information about the normal class and attack types.

This research uses 10 classes, namely *Benign*, *Backdoor*, *DDoS*, *DoS*, *Injection*, *MITM*, *Password*, *Ransomware*, *Scanning*, and *XSS*. The class distribution in the dataset is imbalanced. The *Benign* and *XSS* classes have much larger numbers of samples than the other classes, while classes such as *DoS*, *DDoS*, and *MITM* have very small numbers of samples. This condition becomes the basis for using *macro*-based evaluation metrics and testing *class imbalance* handling scenarios.

Table 3.1: Dataset summary

Component	Description
Dataset	CIC-ToN-IoT
Total records	5,351,760 records
Total columns	85 columns
Label column	<i>Attack</i>
Number of classes	10 classes
Classification type	Multiclass classification
Selected input features	67 numerical traffic features, listed in Appendix

Table 3.1 places the dataset scale, target label, number of classes, and selected feature count in one view. The *Attack* column is selected as the target because it provides detailed attack categories. After the feature inspection process, 67 numerical features are used as input for the *Autoencoder*. The feature list is not shown in the main methodology section so that the discussion remains focused on the experimental workflow, but it is provided in the Appendix.

Table 3.2: Class distribution in CIC-ToN-IoT dataset

Class	Number of Samples	Percentage
<i>Benign</i>	2,515,236	46.9983%
<i>XSS</i>	2,149,308	40.1608%
<i>Password</i>	340,208	6.3569%
<i>Injection</i>	277,696	5.1889%
<i>Scanning</i>	36,205	0.6765%
<i>Backdoor</i>	27,145	0.5072%
<i>Ransomware</i>	5,098	0.0953%
<i>MITM</i>	517	0.0097%
<i>DDoS</i>	202	0.0038%
<i>DoS</i>	145	0.0027%

The distribution in Table 3.2 shows an imbalance in the number of samples across classes. *Benign* and *XSS* dominate most of the dataset, while *DoS*, *DDoS*, and *MITM* have very small numbers of samples. This condition is the reason this research does not rely only on *accuracy*, but also uses *macro* metrics and *confusion matrix*. In addition, this imbalance becomes the basis for designing the *class weight*, *upsampling*, and *downsampling* scenarios on the training data.

3.3 Data Preprocessing

Before entering the modeling stage, the dataset is cleaned and transformed into a numerical format that can be processed by the *Autoencoder* and *Light-GBM*. This process includes removing unused columns, checking missing values, checking invalid values, adjusting data types, label *encoding*, data splitting, and normalization. In this research, several processes that were previously separated in the flowchart, such as feature identification, feature screening, and normalization, are combined in the *preprocessing* discussion because all of them are part of data preparation before model training.

Identifier columns such as *Flow ID*, *Src IP*, *Dst IP*, and *Timestamp* are removed because they do not represent general network traffic patterns. These columns may cause the model to learn specific identities that only apply to the dataset. *Quasi-constant* features are also removed because their value variation is very low and they provide little assistance in distinguishing classes.

Invalid values are handled according to the type of issue. Missing values are checked in each feature column. Infinite values such as *inf* and *-inf* are converted into missing values so that they can be handled consistently. If invalid values appear in only a small number of rows and cannot be logically corrected, those rows are removed. If a feature has too many invalid values or does not have sufficient variation, the feature is removed. Missing labels or

labels that do not match the target classes also cause the related data to be unused because the classification target cannot be confirmed.

After the data is cleaned, the *Attack* column is encoded into numerical labels. Data splitting is performed using *stratified split* with an 80:20 ratio. After data splitting, normalization is performed using *MinMaxScaler*. The normalization parameters are calculated only from the training data, then applied to both the training and testing data. This order is used so that the testing data does not contribute to the formation of *preprocessing* parameters.

Table 3.3: Preprocessing strategy

Process	Action	Rationale
Identifier removal	Remove <i>Flow ID</i> , <i>Src IP</i> , <i>Dst IP</i> , and <i>Timestamp</i>	Avoid learning dataset-specific identities
Invalid value handling	Check missing values, infinite values, and invalid numerical entries	Ensure that data can be processed by Autoencoder and LightGBM
Quasi-constant feature removal	Remove features with very low variation	Reduce uninformative attributes
Label encoding	Convert <i>Attack</i> labels into numerical class indices	Prepare labels for multi-class classification
Stratified split	Split data into training and testing sets using an 80:20 ratio	Preserve class proportion in training and testing data
MinMax normalization	Fit scaler on training data and transform both training and testing data	Avoid data leakage and produce consistent feature scale

The strategy in Table 3.3 emphasizes the order of operations before model training. Data inspection and cleaning are performed before data splitting, while normalization is performed only after the split has been made. In this way, normalization parameters are obtained from the training data, and the testing data remains unseen during the formation of *preprocessing* parameters.

3.4 Dimensionality Reduction Using Autoencoder

Autoencoder is used as a dimensionality reduction method to generate latent feature representations from the original features. This model consists of two main parts, namely the *encoder* and the *decoder*. The *encoder* transforms input data into a lower-dimensional representation, while the *decoder* reconstructs the data from that latent representation. In IDS research, *Autoencoder* is widely used to form new feature representations from high-dimensional data and capture non-linear relationships among features [5, 6, 23].

In this research, the *Autoencoder* receives 67 features from the *preprocessing* stage as input. The architecture used is symmetrical. The *encoder* part reduces the features from 67 neurons to 64 neurons, then 32 neurons, until reaching the *latent layer* with 16 neurons. The *decoder* part reconstructs the representation from 16 neurons to 32 neurons, 64 neurons, and finally back to 67 neurons. The representation in the *latent layer* is used as new features for classification using *LightGBM*.

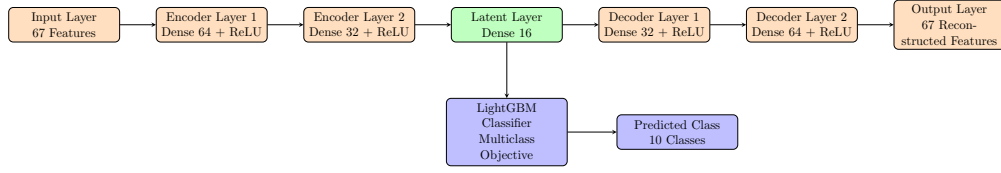


Figure 3.2: Proposed Autoencoder and LightGBM architecture

Figure 3.2 shows the relationship between the *Autoencoder* and *LightGBM*. The main *Autoencoder* flow moves from 67 input features to the *latent layer* with 16 neurons, then reconstructs the representation back into 67 output features. During training, the *decoder* is used so that the model learns to produce a latent representation that can still reconstruct the input features. After training is completed, the part used for classification is the *encoder*. The latent features from the *latent layer* become the input for *LightGBM* to predict 10 intrusion classes.

Table 3.4: Proposed Autoencoder architecture

Component	Specification
Input layer	67 neurons
Encoder layer 1	Dense 64, ReLU activation
Encoder layer 2	Dense 32, ReLU activation
Latent layer	Dense 16, ReLU activation
Decoder layer 1	Dense 32, ReLU activation
Decoder layer 2	Dense 64, ReLU activation
Output layer	67 neurons, Sigmoid activation
Loss function	Mean Squared Error
Optimizer	Adam

The details in Table 3.4 show the compression flow from 67 features into 16 latent features. Reduction is performed gradually so that the representation is not compressed too abruptly. The latent layer with 16 neurons becomes the main output used by *LightGBM*. ReLU activation is used in the hidden layers to capture non-linear relationships, while sigmoid is used in the output layer because the features have been normalized.

3.5 Latent Feature Extraction

After the *Autoencoder* has been trained, latent features are extracted using the *encoder* part. The normalized training data is transformed into latent features to train *LightGBM*. The testing data is also transformed using the same *encoder* so that the feature structure in the testing data remains consistent with the training data.

The resulting latent representation has 16 dimensions. The number of features entering *LightGBM* becomes smaller than the number of original features. The latent features are expected to retain the main patterns from the original data, but in a more compact form.

Table 3.5: Feature transformation across modeling stages

Stage	Feature Dimension	Description
Selected input features	67 features	Numerical traffic features after preprocessing
Normalized features	67 features	Features transformed using MinMaxScaler
Latent features	16 features	Compressed representation generated by the encoder
LightGBM input	16 features	Latent features used for multiclass classification
Predicted output	10 classes	Predicted intrusion class labels

3.6 Experimental Scenarios

This research compares several *class imbalance* handling scenarios to examine their effect on multiclass classification performance. The CIC-ToN-IoT dataset has an imbalanced class distribution, so the model needs to be tested not only under the original condition, but also under several treatments applied to the training data distribution. All scenarios still use the same main workflow, namely the *Autoencoder* as the latent feature extractor and *LightGBM* as the classification model. The difference between scenarios lies in the treatment applied to the latent features of the training data before the *LightGBM* training process.

Table 3.6: Experimental scenarios for class imbalance handling

Scenario	Imbalance Handling	Purpose
S1	Without imbalance handling	Used as the baseline scenario to observe model performance on the original training distribution.
S2	Class weight	Used to reduce bias toward majority classes by assigning larger weights to minority classes during <i>LightGBM</i> training.
S3	Upsampling minority classes	Used to observe whether increasing the number of minority-class samples in the training data improves macro recall and macro F1-score.
S4	Downsampling majority classes	Used to observe whether reducing majority-class dominance in the training data improves the balance of multiclass classification performance.

In scenario S1, the model is trained using the original training data distribution without any special treatment for *class imbalance*. This scenario is used as the baseline comparison to observe the initial model performance. In scenario S2, class weights are calculated based on the number of samples in each class in the training data. Classes with fewer samples receive larger weights so that errors in minority classes receive more attention during *LightGBM* training.

In scenario S3, the number of minority-class samples in the training data is increased through *upsampling*. This approach is used to observe whether adding samples to minority classes can help the model recognize classes that previously had very small amounts of data. In scenario S4, the number of majority-class samples in the training data is reduced through *downsampling*. This approach is used to reduce the dominance of majority classes during the training process.

All *class imbalance* treatments are applied only to the training data. The testing data does not undergo *upsampling*, *downsampling*, or any class distribution modification. This is done so that the evaluation results still reflect model performance on the original distribution of the CIC-ToN-IoT dataset.

3.7 Classification Using *LightGBM*

LightGBM is used as the multiclass classification model in this research. The model is trained using latent features extracted by the *Autoencoder*, not directly using the original features. Therefore, *LightGBM* receives 16 latent

features as input and produces predictions for 10 classes. In IDS research, *LightGBM* is often used because it is efficient for tabular data, can be combined with feature reduction techniques, and is suitable for classification scenarios with many features [8, 9, 15]. The use of *LightGBM* is also relevant for multiclass IDS evaluation and imbalanced data [10].

The *LightGBM* configuration is adjusted to the multiclass classification scenario. The *objective* parameter is set to *multiclass*, while *num_class* is adjusted to the number of classes in the dataset, namely 10 classes. In certain scenarios, the *class_weight* parameter is used to assign different weights to each class.

Table 3.7: Main LightGBM configuration

Parameter	Value	Description
objective	multiclass	Used for multiclass classification
num_class	10	One benign class and nine attack classes
input features	16 latent features	Features extracted from the Autoencoder latent layer
learning_rate	0.05 or 0.1	Initial value for model training
n_estimators	100 or 200	Number of boosting iterations
num_leaves	31	Controls tree complexity
max_depth	-1 or tuned value	Controls maximum tree depth
class_weight	balanced or custom	Used in the class weight scenario
random_state	42	Used for reproducibility

3.8 Evaluation and Result Analysis

Evaluation is performed by comparing the predicted labels and the actual labels on the testing data. The metrics used include *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*. *Accuracy* is used to observe the overall proportion of correct predictions, while *macro precision*, *macro recall*, and *macro F1-score* are used to examine the performance of each class with equal weighting. This approach is suitable for multiclass intrusion classification because class distributions are often imbalanced [2, 14, 10].

Table 3.8: Evaluation metrics

Metric	Purpose
Accuracy	Measures the overall proportion of correct predictions
Macro precision	Measures average prediction precision across all classes with equal class weight
Macro recall	Measures average detection ability across all classes with equal class weight
Macro F1-score	Measures the balance between macro precision and macro recall
Confusion matrix	Shows detailed correct and incorrect predictions for each class

The metrics in Table 3.8 are selected so that evaluation does not rely only on *accuracy*. *Macro precision* shows prediction precision for each class, *macro recall* shows the model’s ability to recognize data from each class, and *macro F1-score* combines both into a single value. The *confusion matrix* is used to observe error patterns across classes.

3.9 Algorithm

Algorithm 1 shows the core workflow of the proposed method. The dataset loading, feature inspection, *preprocessing*, data splitting, and normalization stages have been explained in the previous sections, so this algorithm focuses on the main process after the data is ready to be used. This process includes *Autoencoder* training, latent feature extraction, application of *class imbalance* handling scenarios, *LightGBM* training, intrusion class prediction, and result evaluation.

Algorithm 1 Core Autoencoder and LightGBM algorithm for multiclass intrusion classification

```

1: procedure AE_LIGHTGBM_CORE( $X_{train}^{norm}, X_{test}^{norm}, y_{train}, y_{test}, C, S$ )
2:   Input: normalized training features  $X_{train}^{norm}$ , normalized testing features  $X_{test}^{norm}$ , training labels  $y_{train}$ , testing labels  $y_{test}$ , number of classes  $C$ , and imbalance scenario  $S$ 
3:   Output: accuracy, macro precision, macro recall, macro F1-score, and confusion matrix
4:   Initialize Autoencoder parameters  $\theta = \{W_e, b_e, W_d, b_d\}$ 
5:   Train the Autoencoder by minimizing reconstruction loss:
6:      $\theta^* \leftarrow \arg \min_{\theta} \|X_{train}^{norm} - \text{Decoder}_{\theta}(\text{Encoder}_{\theta}(X_{train}^{norm}))\|_2^2$ 
7:   Extract latent features using the trained encoder:
8:      $Z_{train} \leftarrow \text{Encoder}_{\theta^*}(X_{train}^{norm})$ 
9:      $Z_{test} \leftarrow \text{Encoder}_{\theta^*}(X_{test}^{norm})$ 
10:  if  $S = S1$  then
11:    Use  $Z_{train}$  and  $y_{train}$  without imbalance handling
12:  else if  $S = S2$  then
13:    Compute class weights:
14:     $n \leftarrow |y_{train}|$ 
15:    for each class  $c \in \{1, 2, \dots, C\}$  do
16:       $n_c \leftarrow \sum_{i=1}^n \mathbb{I}(y_i = c)$ 
17:       $w_c \leftarrow \frac{n}{C \cdot n_c}$ 
18:    end for
19:  else if  $S = S3$  then
20:    Apply upsampling to minority classes on  $Z_{train}$  and  $y_{train}$ 
21:  else if  $S = S4$  then
22:    Apply downsampling to majority classes on  $Z_{train}$  and  $y_{train}$ 
23:  end if
24:  Train the LightGBM classifier using latent training features:
25:     $M_{LGBM} \leftarrow \text{TrainLightGBM}(Z_{train}, y_{train}, S)$ 
26:  Predict intrusion classes on latent testing features:
27:     $\hat{y}_{test} \leftarrow M_{LGBM}(Z_{test})$ 
28:  Evaluate prediction results:
29:     $R \leftarrow \text{Evaluate}(y_{test}, \hat{y}_{test})$ 
30:  return  $R$ 
31: end procedure

```

Algorithm 1 receives normalized training and testing data, namely X_{train}^{norm} and X_{test}^{norm} . The training and testing labels are denoted by y_{train} and y_{test} . The number of classes is denoted by C , while the *class imbalance* handling scenario is denoted by S . With this arrangement, the algorithm no longer includes detailed *preprocessing* steps because that process has been explained

in the *Data Preprocessing* section.

The initial lines of the algorithm show the initialization of the *Autoencoder* parameters, namely $\theta = \{W_e, b_e, W_d, b_d\}$. These parameters are then updated through training by minimizing the reconstruction error. This process produces the best parameter θ^* , which is used by the *encoder* to extract latent features from the training and testing data. The extraction results are denoted by Z_{train} and Z_{test} .

After the latent features are obtained, the algorithm applies a *class imbalance* handling scenario. In scenario S1, the training data is used without changing the class distribution. In scenario S2, class weights are calculated based on the number of data in each class. In scenario S3, the number of minority-class samples in the training data is increased through *upsampling*. In scenario S4, the number of majority-class samples in the training data is reduced through *downsampling*. All scenarios are applied only to the training data, while the testing data keeps its original distribution.

The next stage is *LightGBM* training using latent features Z_{train} and labels y_{train} . The resulting model is denoted by M_{LGBM} . The model is then used to predict classes on the latent features of the testing data Z_{test} . The prediction result $\hat{y}_{test}$ is compared with the actual label y_{test} to produce evaluation values. Evaluation is performed using *accuracy*, *macro precision*, *macro recall*, *macro F1-score*, and *confusion matrix*.

References

- [1] Mohanad Sarhan, Siamak Layeghy, and Marius Portmann. Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection. *Big Data Research*, 30:100359, 2022.
- [2] Huan Ma, Wan Zhang, Dalong Zhang, and Baozhan Chen. An IoT intrusion detection framework based on feature selection and large language models fine-tuning. *Scientific Reports*, 15, 2025.
- [3] Mohamed Aly Bouke and Azizol Abdullah. An empirical study of pattern leakage impact during data preprocessing on machine learning-based intrusion detection models reliability. *Expert Systems with Applications*, 230:120715, 2023.
- [4] Liam Daly Manocchio, Siamak Layeghy, Marcus Gallagher, and Marius Portmann. An empirical evaluation of preprocessing methods for machine learning based network intrusion detection systems. *Engineering Applications of Artificial Intelligence*, 158:111289, 2025.
- [5] Tianyue Zhang, Wei Chen, Yuxiao Liu, and Lifa Wu. An intrusion detection method based on stacked sparse autoencoder and improved gaussian mixture model. *Computers & Security*, 128:103144, 2023.
- [6] Jing Li, Hewan Chen, Mohd Shahizan Othman, and Lizawati Mi Yusuf. Enhancing IoT security: A comparative study of feature reduction techniques for intrusion detection system. *Intelligent Systems with Applications*, 23:200407, 2024.
- [7] M. V. Hari Vinayak and T. Jarin. A hybrid model for detecting intrusions using stacked autoencoders and extreme gradient boosting. *Computers & Security*, 150:104212, 2025.
- [8] Jingmei Liu, Yuanbo Gao, and Fengjie Hu. A fast network intrusion detection system using adaptive synthetic oversampling and LightGBM. *Computers & Security*, 106:102289, 2021.

- [9] Adeel Munawar, Muhammad Nadeem Ali, Awais Qasim, and Byung-Seo Kim. GWO-LightGBM: A hybrid grey wolf optimized light gradient boosting model for cyber-physical system security. *Computer Modeling in Engineering & Sciences*, 145(1), 2025.
- [10] Abubakar Wakili and Sara Bakkali. A resilient IoT intrusion detection system using hybrid feature selection and explainable ensemble learning. *Results in Engineering*, 28:107392, 2025.
- [11] Md Mahbubur Rahman, Shaharia Al Shakil, and Mizanur Rahman Mustakim. A survey on intrusion detection system in IoT networks. *Cyber Security and Applications*, 3:100082, 2025.
- [12] Fouad Al Tfaily, Zakariya Ghalmane, Mohamed El Amine Brahmia, Hussein Hazimeh, Ali Jaber, and Mourad Zghal. Graph-based federated learning approach for intrusion detection in IoT networks. *Scientific Reports*, 15:41264, 2025.
- [13] Jing Li, Mohd Shahizan Othman, Hewan Chen, and Lizawati Mi Yusuf. Optimizing IoT intrusion detection system: feature selection versus feature extraction in machine learning. *Journal of Big Data*, 11:36, 2024.
- [14] Muhammad Farhan, Hafiz Waheed ud din, Saadat Ullah, Muhammad Sajjad Hussain, Muhammad Amir Khan, Tehseen Mazhar, Umar Farooq Khattak, and Ines Hilali Jaghdam. Network-based intrusion detection using deep learning technique. *Scientific Reports*, 15:25550, 2025.
- [15] Brou Médard Kouassi, Abou Bakary Ballo, Kacoutchy Jean Ayikpa, Diarra Mamadou, and Minfonga Zié Jérôme Coulibaly. Top-k feature selection for IoT intrusion detection: Contributions of XGBoost, LightGBM, and random forest. *Future Internet*, 17(11):529, 2025.
- [16] Syed Rizwan Hassan, Muhammad Usama Tanveer, Sunil Prajapat, and Mohammad Shabaz. A comprehensive survey on intrusion detection in internet of medical things: Datasets, federated learning, blockchain, and future research directions. *ICT Express*, 11:1291–1310, 2025.
- [17] Omar Achbarou, Toufik Datsi, Outmane Bourkougou, and Ahmed My El kiram. Enhanced intrusion detection system using feature selection and hybrid learning models for high performance and efficiency in an IoT environment. *Journal of Engineering Research*, 14:953–965, 2026.
- [18] Abdullah Alfahaid, Easa Alalwany, Abdulqader M. Almars, Fatemah Alharbi, Elsayed Atlam, and Imad Mahgoub. Machine learning-based security solutions for IoT networks: A comprehensive survey. *Sensors*, 25(11):3341, 2025.

- [19] K. Swathi and G. Hima Bindu. An automated intrusion detection system in IoT system using attention based deep bidirectional sparse auto encoder model. *Knowledge-Based Systems*, 305:112633, 2024.
- [20] Muaadh A. Alsoufi, Maheyzah Md Siraj, Fuad A. Ghaleb, Muna Al-Razgan, Mahfoudh Saeed Al-Asaly, Taha Alfakih, and Faisal Saeed. Anomaly-based intrusion detection model using deep learning for IoT networks. *Computer Modeling in Engineering & Sciences*, 141(1), 2024.
- [21] Kuburat Oyeranti Adefemi, Murimo Bethel Mutanga, and Oyeniyi Akeem Alimi. A hybrid CNN–GRU deep learning model for IoT network intrusion detection. *Journal of Sensor and Actuator Networks*, 14(5):96, 2025.
- [22] Aya G. Ayad, Mostafa M. El-Gayar, Noha A. Hikal, and Nehal A. Sakr. Efficient real-time anomaly detection in IoT networks using one-class autoencoder and deep neural network. *Electronics*, 14(1):104, 2025.
- [23] Wenbin Yao, Longcan Hu, Yingying Hou, and Xiaoyong Li. A lightweight intelligent network intrusion detection system using one-class autoencoder and ensemble learning for IoT. *Sensors*, 23(8):4141, 2023.
- [24] Yiran Xiao, Yaokai Feng, and Kouichi Sakurai. An efficient detection mechanism of network intrusions in IoT environments using autoencoder and data partitioning. *Computers*, 13(10):269, 2024.
- [25] Sarah Bin Hulayyil, Shancang Li, and Neetesh Saxena. Explainable AI-based intrusion detection in IoT systems. *Internet of Things*, 31:101589, 2025.

Appendices